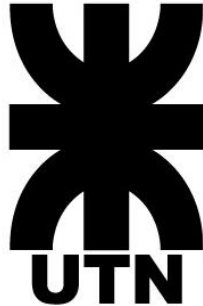


Universidad Tecnológica Nacional



Tecnicatura Universitaria en Programación a Distancia

Trabajo Práctico Integrador — Programación II
Food Store — Sistema de Gestión de Pedidos de Comida

Alumna: Martínez, Victoria Abigail (vickymiley30@gmail.com)

Comisión: 8

Profesor: Juan Cruz Robledo

Fecha: Junio 22 del 2026

Índice

1. Introducción.....	3
2. Marco Teórico.....	4
3. Decisiones Técnicas y Arquitectura.....	5
4. Dificultades y Soluciones.....	6
5. Bibliografía / Webgrafía.....	7
6. Links.....	8
7. Capturas del Sistema en Funcionamiento.....	9

Introducción

El presente trabajo práctico integrador consiste en el desarrollo de **Food Store**, un sistema de gestión de pedidos de comida implementado en Java 21 con NetBeans. El sistema permite gestionar categorías, productos, usuarios y pedidos desde un menú de consola, aplicando los principios de la Programación Orientada a Objetos.

Marco Teórico

El presente trabajo fue desarrollado en **Java 21** utilizando el entorno de desarrollo **NetBeans**. Se aplicaron los siguientes conceptos de Programación Orientada a Objetos:

- **Herencia:** todas las entidades del sistema (Categoria, Producto, Usuario, Pedido, DetallePedido) heredan de la clase abstracta Base, que centraliza los atributos comunes id, eliminado y createdAt.
- **Interfaces:** se implementó la interfaz Calculable con el método calcularTotal(), que es implementada por la clase Pedido para calcular el total de cada pedido.
- **Polimorfismo:** las entidades sobrescriben el método toString() para mostrar su información de forma personalizada en consola.
- **Colecciones:** se utilizaron ArrayList para almacenar en memoria las listas de categorías, productos, usuarios y pedidos durante la ejecución del programa.
- **Excepciones personalizadas:** se crearon EntidadNoEncontradaException y StockInvalidoException para manejar errores específicos del dominio.
- **Enumerados (Enum):** se utilizaron Rol, Estado y FormaPago para representar valores constantes y evitar ambigüedades.

Decisiones Técnicas y Arquitectura

El proyecto está organizado en los siguientes paquetes:

- **foodstore.entities** → contiene las clases del modelo (Base, Categoria, Producto, Usuario, Pedido, DetallePedido) y la interfaz Calculable.
- **foodstore.enums** → contiene los enumerados Rol, Estado y FormaPago.
- **foodstore.exception** → contiene las excepciones personalizadas EntidadNoEncontradaException y StockInvalidoException.
- **foodstore.service** → contiene la lógica de negocio (CategoriaService, ProductoService, UsuarioService, PedidoService).
- **foodstore.ui** → contiene el menú principal y los submenús de consola (Menu) y la clase Main.

Decisiones tomadas:

- Se utilizó **soft delete** (eliminación lógica) en todas las entidades, marcando el campo `eliminado = true` en vez de borrar físicamente el objeto de la colección, para mantener el historial.
- El **mail del usuario** es único, validado recorriendo la colección antes de crear o editar.
- El **precio y stock** de los productos no pueden ser negativos, controlado mediante `StockInvalidoException`.
- La interfaz **Calculable** con el método `calcularTotal()` es implementada por `Pedido`, que suma los subtotales de todos sus detalles.

Dificultades y Soluciones

Durante el desarrollo del proyecto surgieron los siguientes inconvenientes:

- **Problema con los imports:** al renombrar los paquetes durante el desarrollo, algunos imports no se actualizaron automáticamente y generaron errores de compilación. Se resolvió revisando y corrigiendo manualmente cada import en las clases afectadas.
- **Error en clases con clase anidada:** en una de las clases se escribió accidentalmente una clase dentro de otra, lo que generó un error de compilación. Se resolvió eliminando la clase anidada y dejando solo la clase correcta.
- **Encoding en consola:** los caracteres especiales como acentos y la ñ se mostraban incorrectamente en la consola de NetBeans. Es un problema de configuración del entorno, no del código en sí.
- **Interfaz Calculable:** el método calcularTotal() debía ser void según el UML del enunciado, por lo que el total se almacena como atributo del Pedido y se actualiza llamando al método cada vez que se agrega un detalle.
- **Error en interfaz PagoConDescuento:** la interfaz PagoConDescuento no extendía correctamente a Pago, lo que generaba que el método procesarPago() no fuera reconocido como implementación válida en TarjetaCredito y PayPal. Se resolvió agregando `extends Pago` en la declaración de la interfaz.
- **Pérdida de datos al reiniciar:** al cerrar y volver a ejecutar el programa, todos los datos cargados se perdían porque el sistema almacena la información en memoria mediante colecciones ArrayList. Se tuvo que volver a cargar los datos en cada sesión de prueba. Esto es una característica del sistema, ya que la consigna no requiere persistencia en base de datos.

Bibliografía / Webgrafía

Material de cátedra — Unidad 6:

- Robledo, J. C. (2026). *Apunte: Enums en Java*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Robledo, J. C. (2026). *Complemento Práctico: Algoritmos de búsqueda y Enums*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Robledo, J. C. (2026). *Complemento Práctico: El Fin de los Arrays Estáticos*. Tecnicatura Universitaria en Programación a Distancia, UTN.

Material de cátedra — Unidad 7:

- Robledo, J. C. (2026). *Control y Generalización*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Robledo, J. C. (2026). *El Arte de No Repetirse*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Robledo, J. C. (2026). *El Arte de la Adaptabilidad*. Tecnicatura Universitaria en Programación a Distancia, UTN.

Material de cátedra — Unidad 8:

- Robledo, J. C. (2026). *Más allá de la Herencia Simple*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Robledo, J. C. (2026). *Evolución de las Interfaces*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Robledo, J. C. (2026). *Control de Daños y Robustez*. Tecnicatura Universitaria en Programación a Distancia, UTN.
- Robledo, J. C. (2026). *Control Total: Lanzamiento y Recursos*. Tecnicatura Universitaria en Programación a Distancia, UTN.

Documentación oficial:

- Oracle Corporation. (s.f.). *Java 21 Documentation*.
<https://docs.oracle.com/en/java/javase/21/>
- Apache NetBeans. (s.f.). *NetBeans IDE Documentation*.
<https://netbeans.apache.org/kb/>

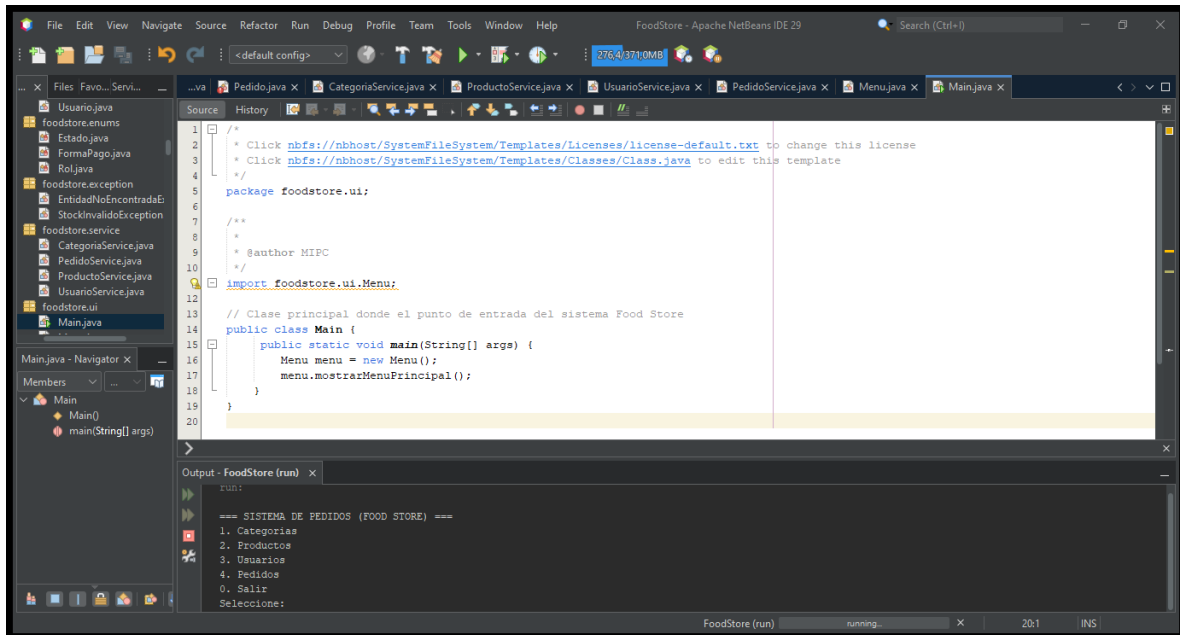
Links

Link al repositorio del Proyecto: <https://github.com/vmartinez22/FoodStore>

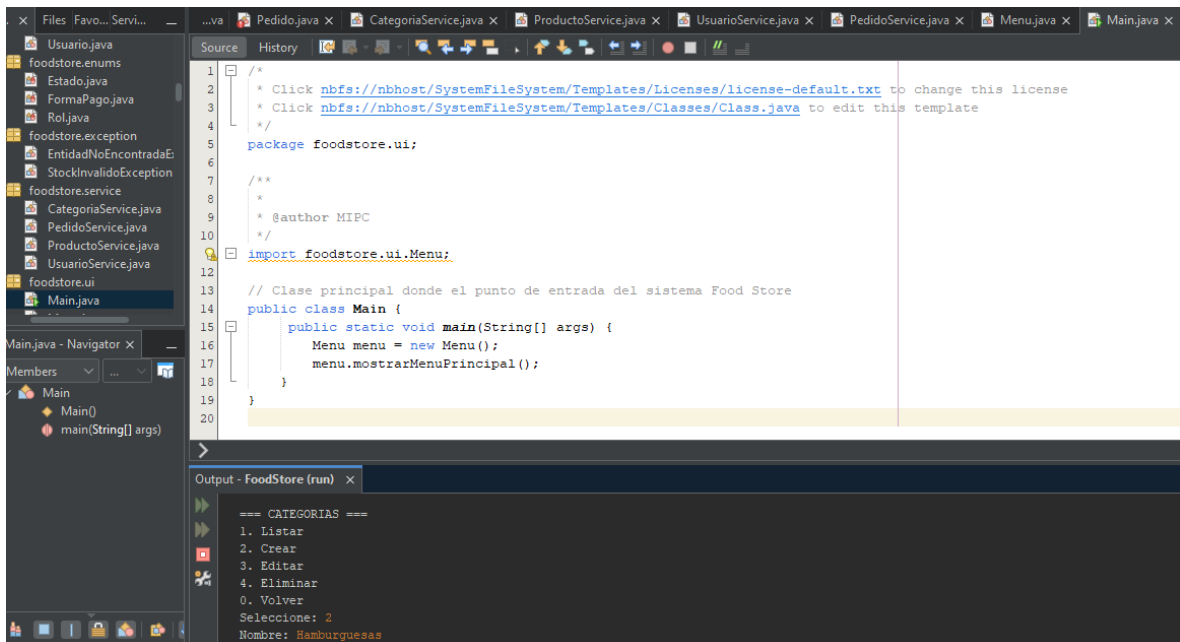
Link al enlace del video explicativo: <https://youtu.be/kf7MHIJbavQ>

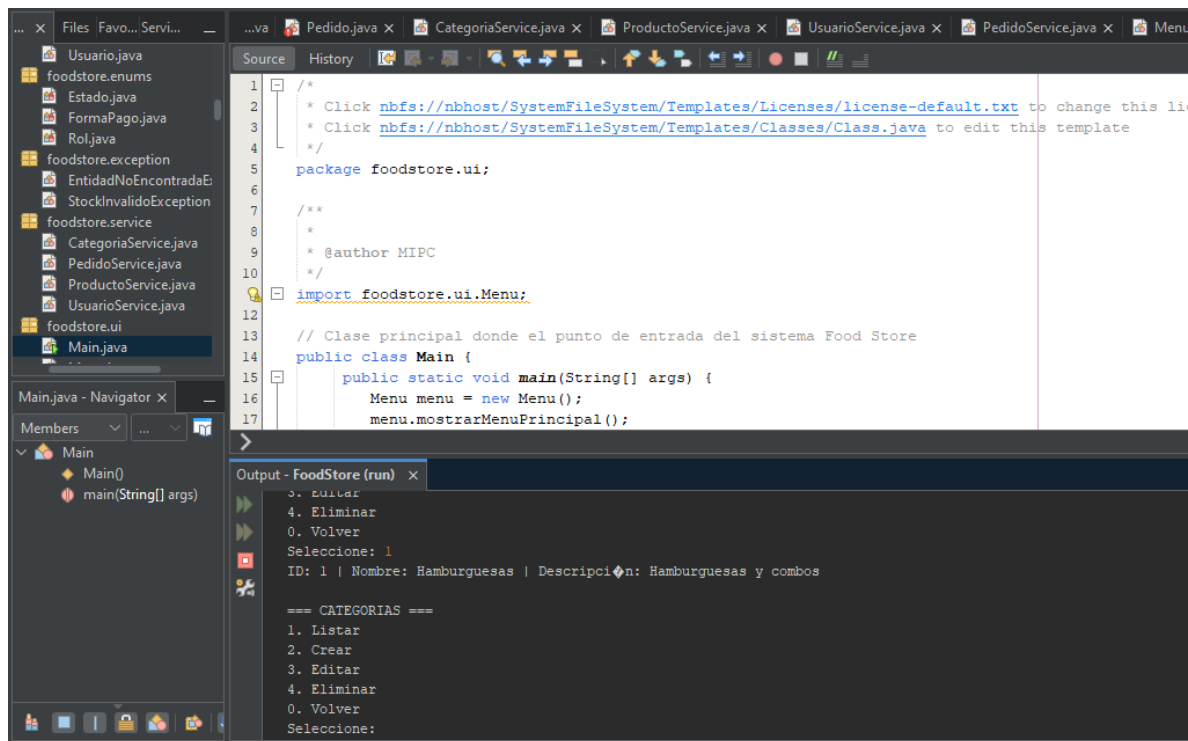
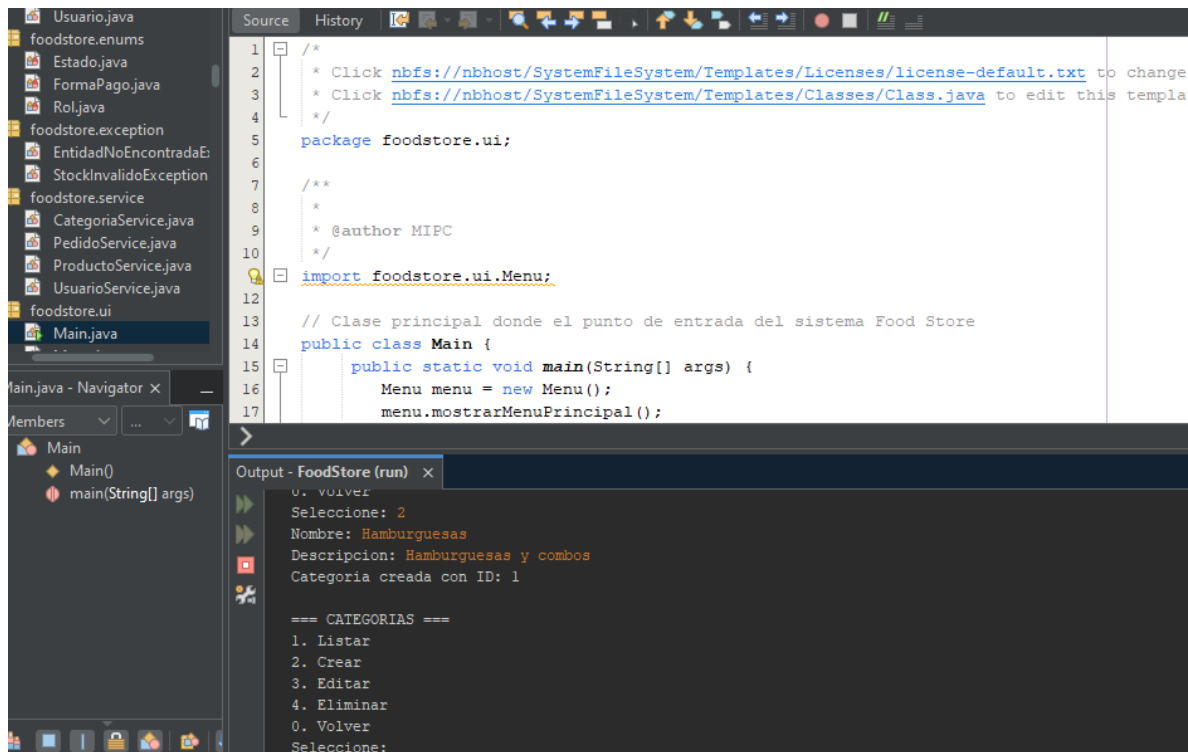
Capturas del Sistema en Funcionamiento

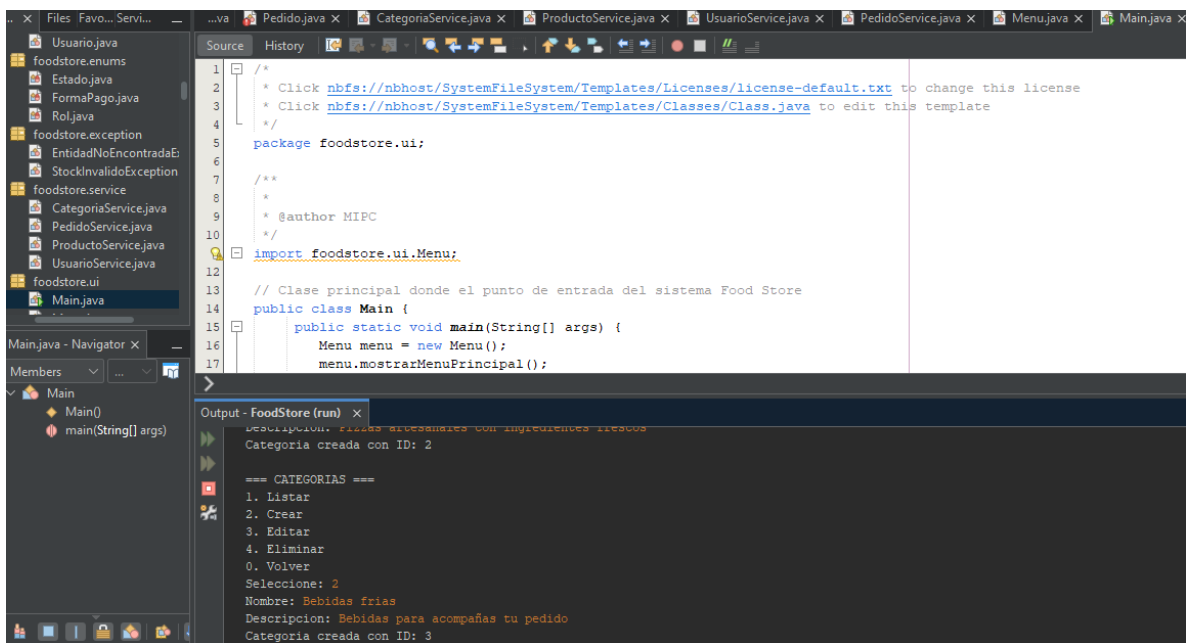
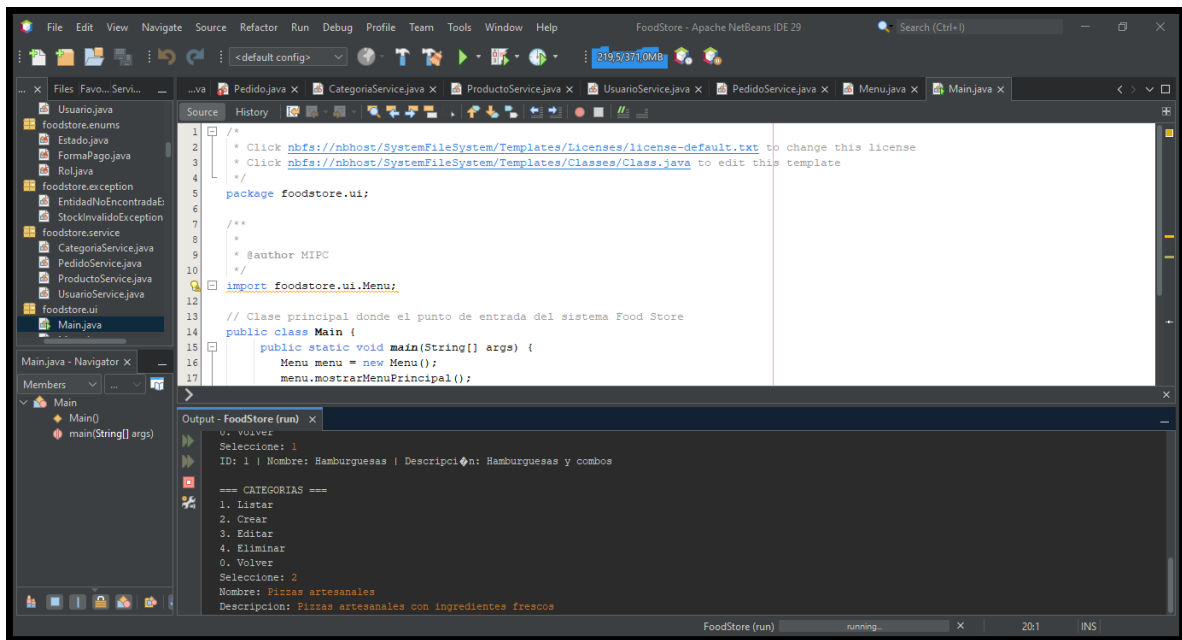
Menú principal



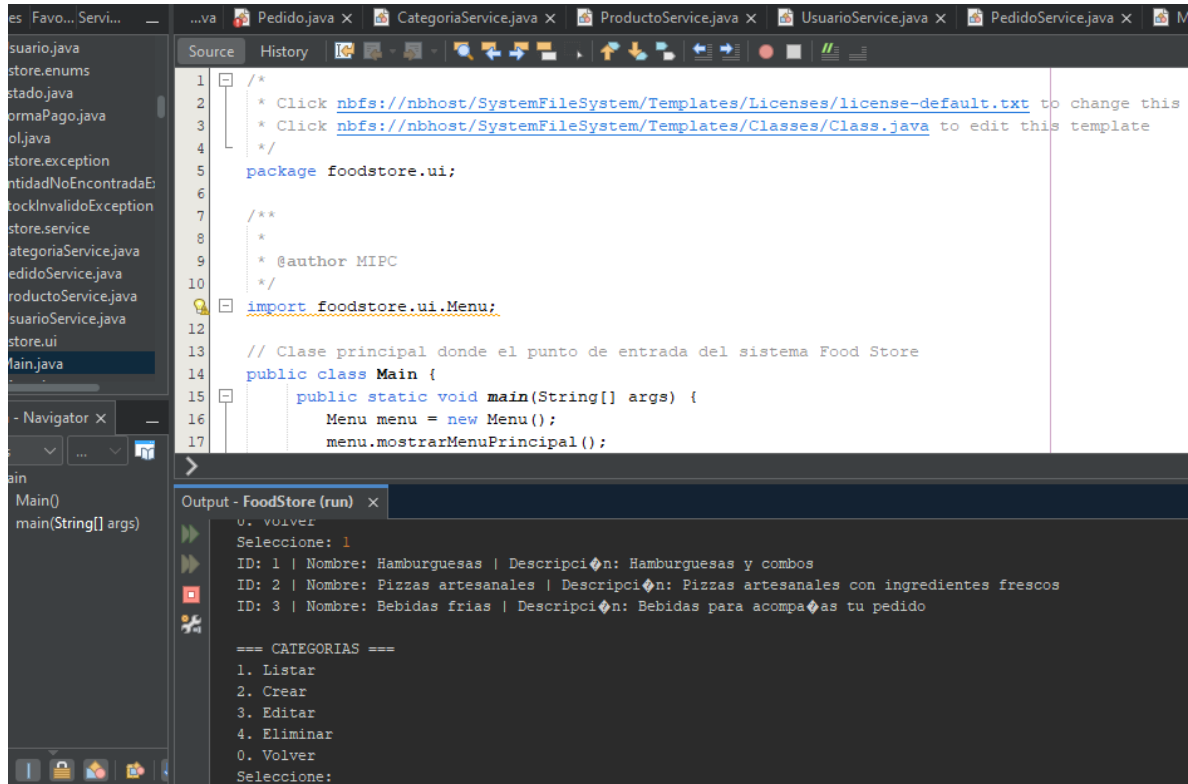
Crear Categorías







Listar categorías



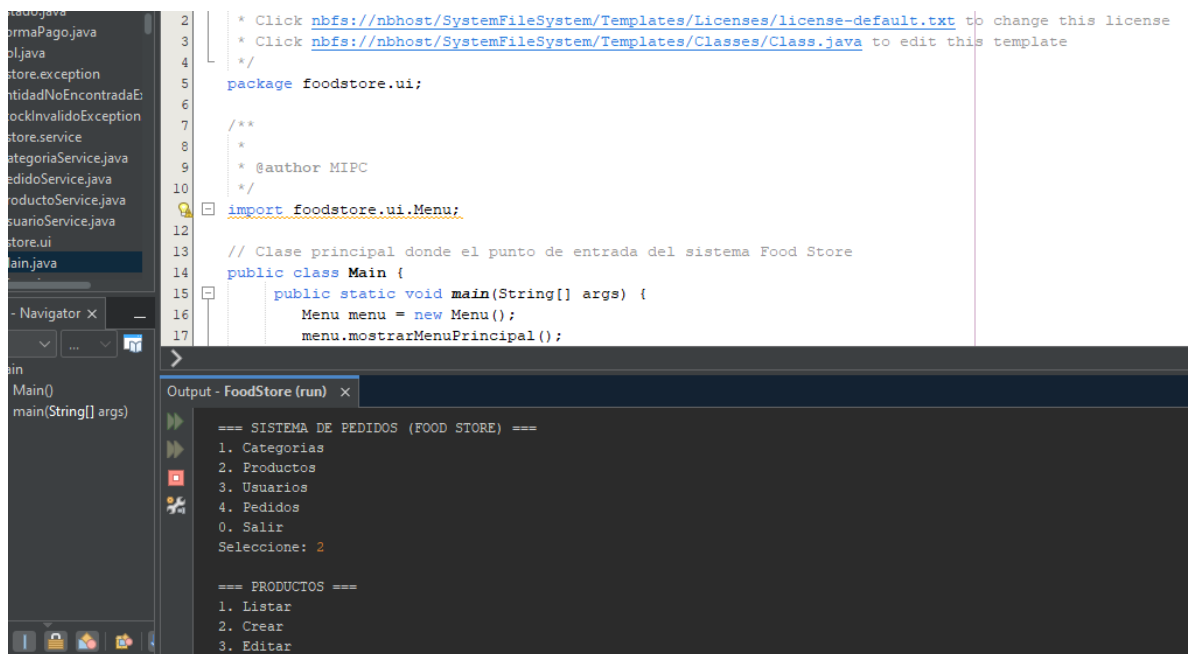
```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5  package foodstore.ui;
6
7  /**
8   *
9   * @author MIPC
10  */
11
12  import foodstore.ui.Menu;
13
14  // Clase principal donde el punto de entrada del sistema Food Store
15  public class Main {
16      public static void main(String[] args) {
17          Menu menu = new Menu();
18          menu.mostrarMenuPrincipal();
19      }
20  }
```

Output - FoodStore (run) x

```
0. volver
Seleccione: 1
ID: 1 | Nombre: Hamburguesas | Descripción: Hamburguesas y combos
ID: 2 | Nombre: Pizzas artesanales | Descripción: Pizzas artesanales con ingredientes frescos
ID: 3 | Nombre: Bebidas frias | Descripción: Bebidas para acompañar tu pedido

=== CATEGORIAS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione:
```

Crear producto

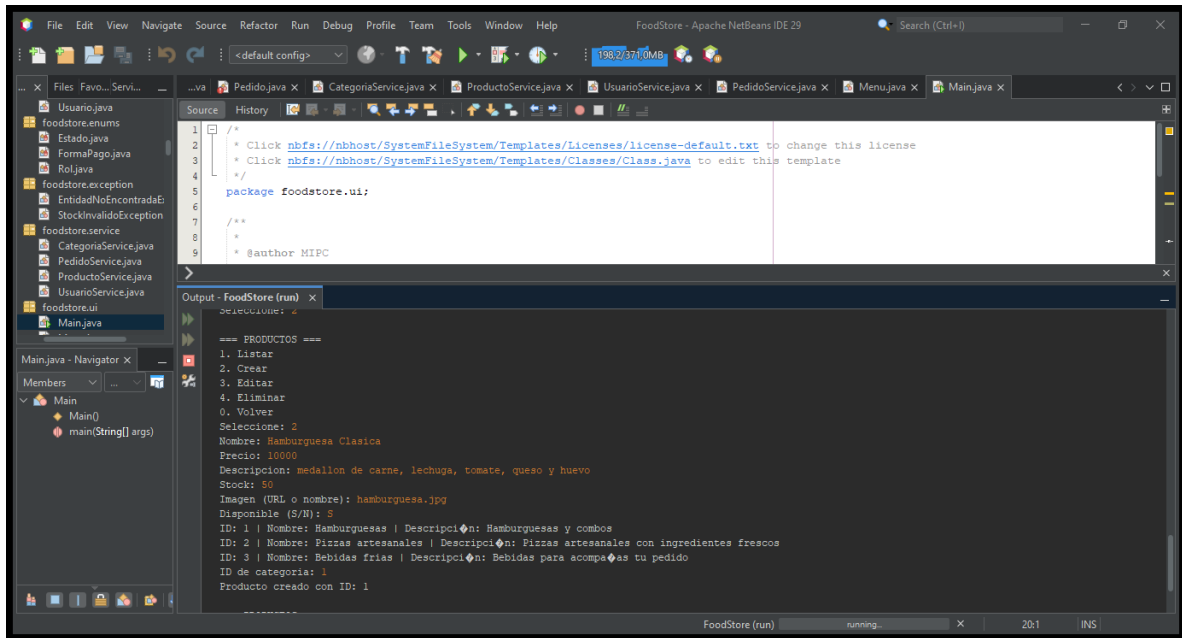


```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5  package foodstore.ui;
6
7  /**
8   *
9   * @author MIPC
10  */
11
12  import foodstore.ui.Menu;
13
14  // Clase principal donde el punto de entrada del sistema Food Store
15  public class Main {
16      public static void main(String[] args) {
17          Menu menu = new Menu();
18          menu.mostrarMenuPrincipal();
19      }
20  }
```

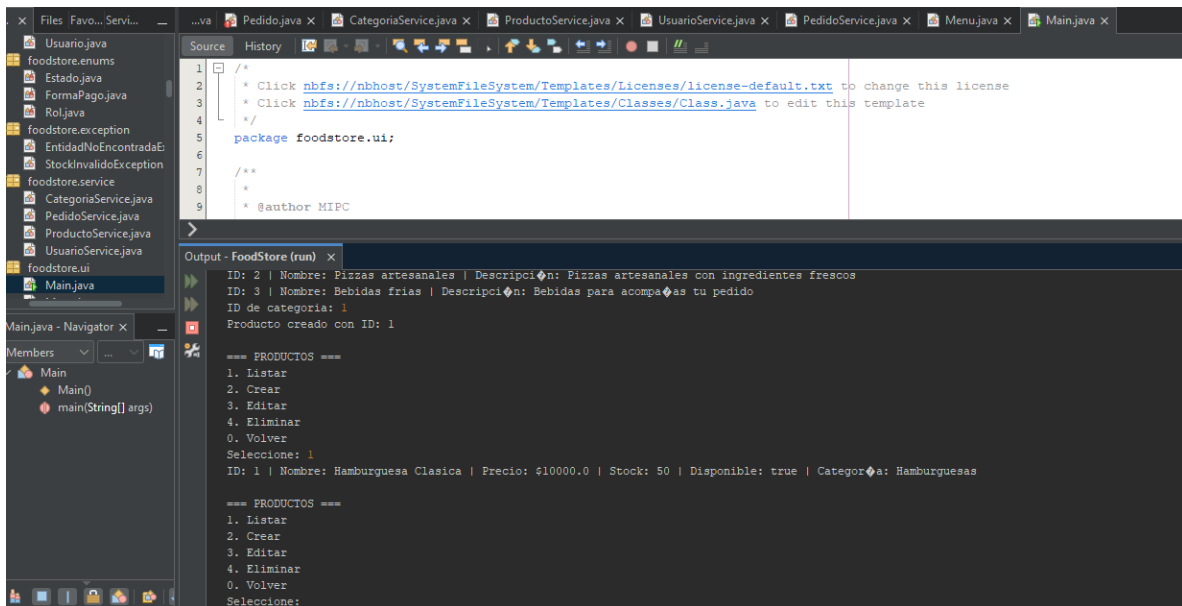
Output - FoodStore (run) x

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorias
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 2

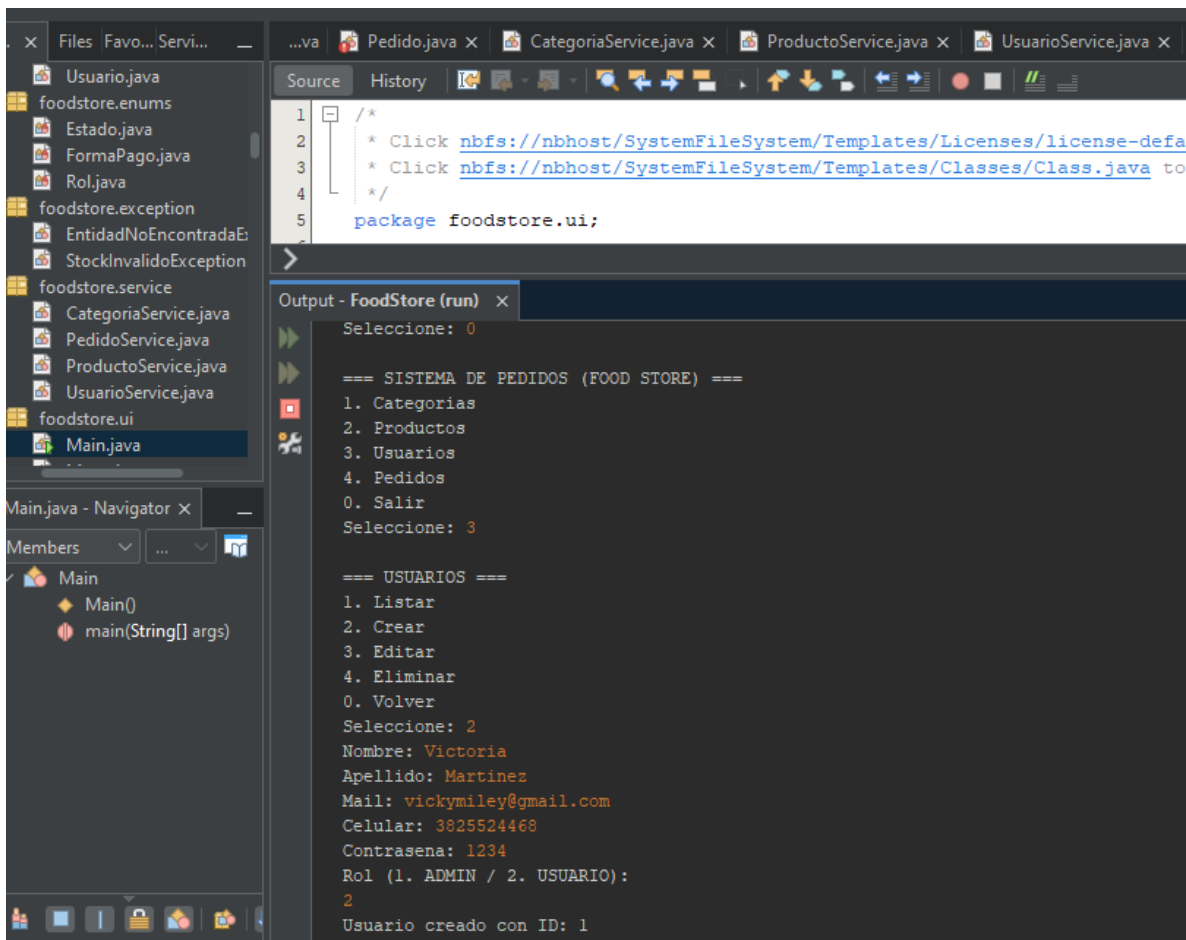
=== PRODUCTOS ===
1. Listar
2. Crear
3. Editar
```



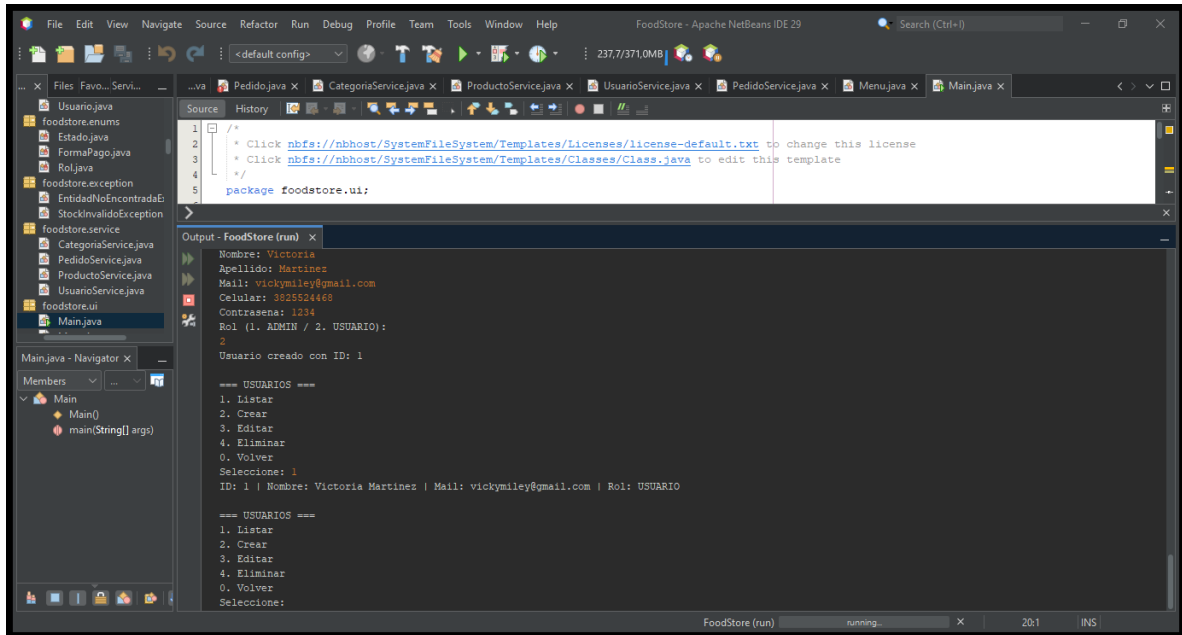
Listar producto



Crear usuario



Listar usuario



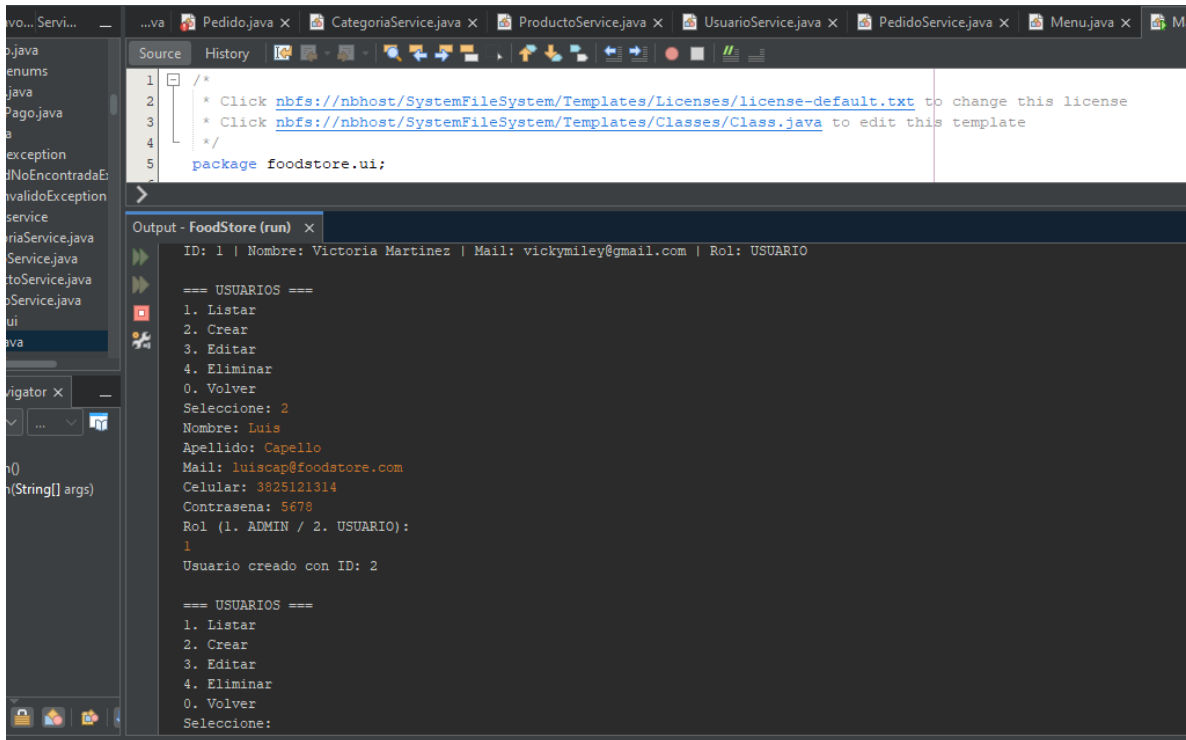
```
1  /*
2  2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  4  */
5  package foodstore.ui;
```

Output - FoodStore (run) X

```
Nombre: Victoria
Apellido: Martinez
Mail: vickymiley@gmail.com
Celular: 3825524468
Contraseña: 1234
Rol (1. ADMIN / 2. USUARIO):
2
Usuario creado con ID: 1

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID: 1 | Nombre: Victoria Martinez | Mail: vickymiley@gmail.com | Rol: USUARIO

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione:
```



```
1  /*
2  2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  4  */
5  package foodstore.ui;
```

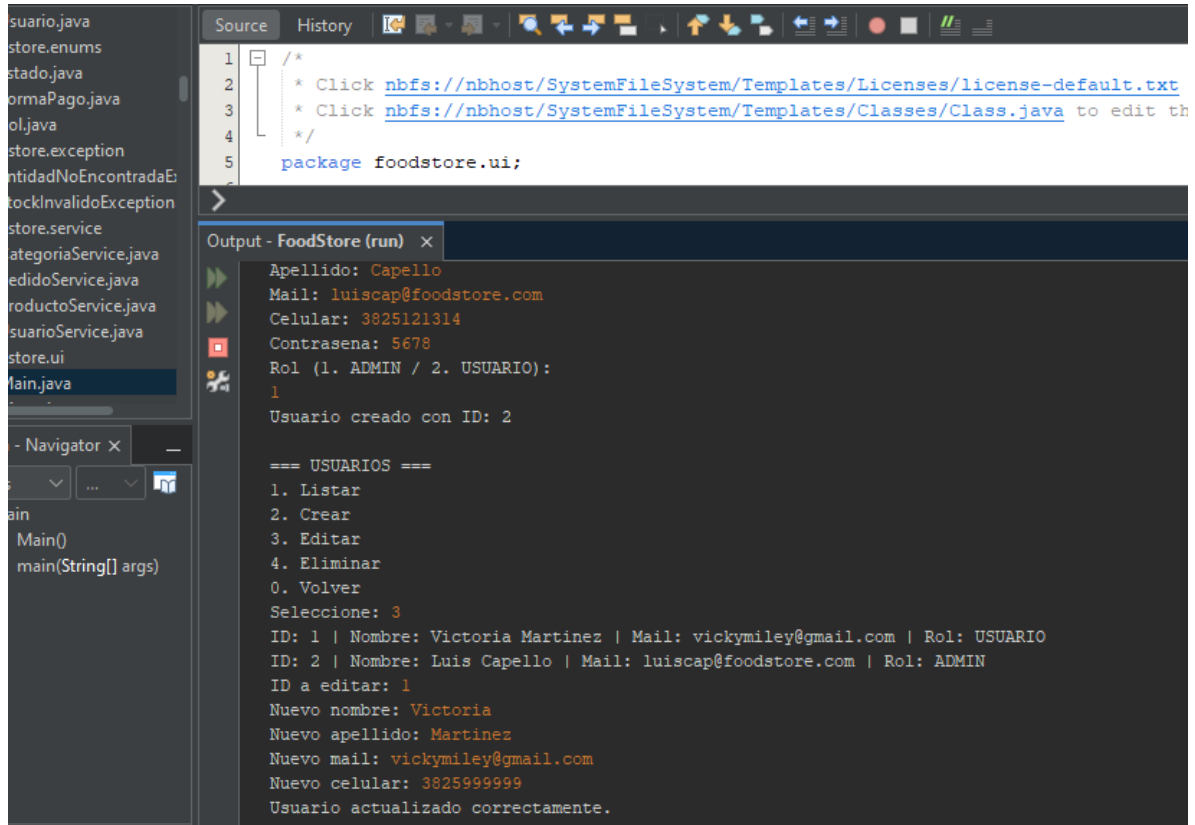
Output - FoodStore (run) X

```
ID: 1 | Nombre: Victoria Martinez | Mail: vickymiley@gmail.com | Rol: USUARIO

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Luis
Apellido: Capello
Mail: luiscap@foodstore.com
Celular: 3825121314
Contraseña: 5678
Rol (1. ADMIN / 2. USUARIO):
1
Usuario creado con ID: 2

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione:
```

Editar usuario



The screenshot shows an IDE with a project explorer on the left, a source editor in the center, and an output console at the bottom.

Source Editor:

```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit th
4  */
5  package foodstore.ui;
```

Output - FoodStore (run):

```
Apellido: Capello
Mail: luisca@foodstore.com
Celular: 3825121314
Contraseña: 5678
Rol (1. ADMIN / 2. USUARIO):
1
Usuario creado con ID: 2

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 3
ID: 1 | Nombre: Victoria Martinez | Mail: vickymiley@gmail.com | Rol: USUARIO
ID: 2 | Nombre: Luis Capello | Mail: luisca@foodstore.com | Rol: ADMIN
ID a editar: 1
Nuevo nombre: Victoria
Nuevo apellido: Martinez
Nuevo mail: vickymiley@gmail.com
Nuevo celular: 3825999999
Usuario actualizado correctamente.
```


Eliminar usuario

```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this li
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5  package foodstore.ui;
```

Output - FoodStore (run) X

```
Usuario actualizado correctamente.

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 4
ID: 1 | Nombre: Victoria Martinez | Mail: vickymiley@gmail.com | Rol: USUARIO
ID: 2 | Nombre: Luis Capello | Mail: luiscap@foodstore.com | Rol: ADMIN
ID a eliminar: 2
Confirma eliminacion? (S/N): S
Usuario eliminado correctamente.

=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
ID: 1 | Nombre: Victoria Martinez | Mail: vickymiley@gmail.com | Rol: USUARIO

=== USUARIOS ===
```

Crear pedido con detalles

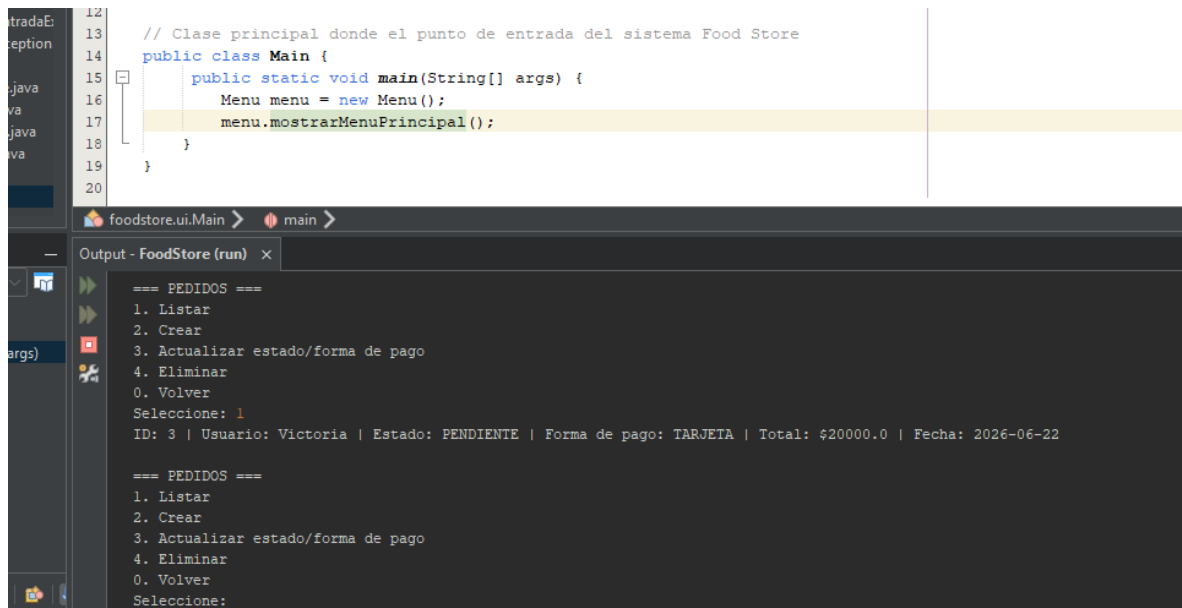
```
8  *
9  * @author MIPC
10 */
11
12 import foodstore.ui.Menu;
13
// Clase principal donde el punto de entrada del sistema Food Store
foodstore.ui.Main > main >
```

Output - FoodStore (run) X

```
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 4

=== PEDIDOS ===
1. Listar
2. Crear
3. Actualizar estado/forma de pago
4. Eliminar
0. Volver
Seleccione: 2
ID: 1 | Nombre: Victoria Martinez | Mail: vickymiley30@gmail.com | Rol: USUARIO
ID de usuario: 1
Forma de pago (1. TARJETA / 2. TRANSFERENCIA / 3. EFECTIVO):
1
Pedido creado con ID: 3
ID: 1 | Nombre: Hamburguesa clasica | Precio: $10000.0 | Stock: 50 | Disponible: true | Categoria: Hamburguesas artesanales
ID de producto: 1
Cantidad: 2
Producto agregado al pedido correctamente.
Agregar otro producto? (S/N): N
Pedido finalizado. Total: $20000.0
```

Listar pedidos



The screenshot shows an IDE with the `foodstore.ui.Main` class open. The class contains a `main` method that creates a `Menu` object and calls `mostrarMenuPrincipal()`. The output window shows the menu options and the result of selecting option 1 (Listar).

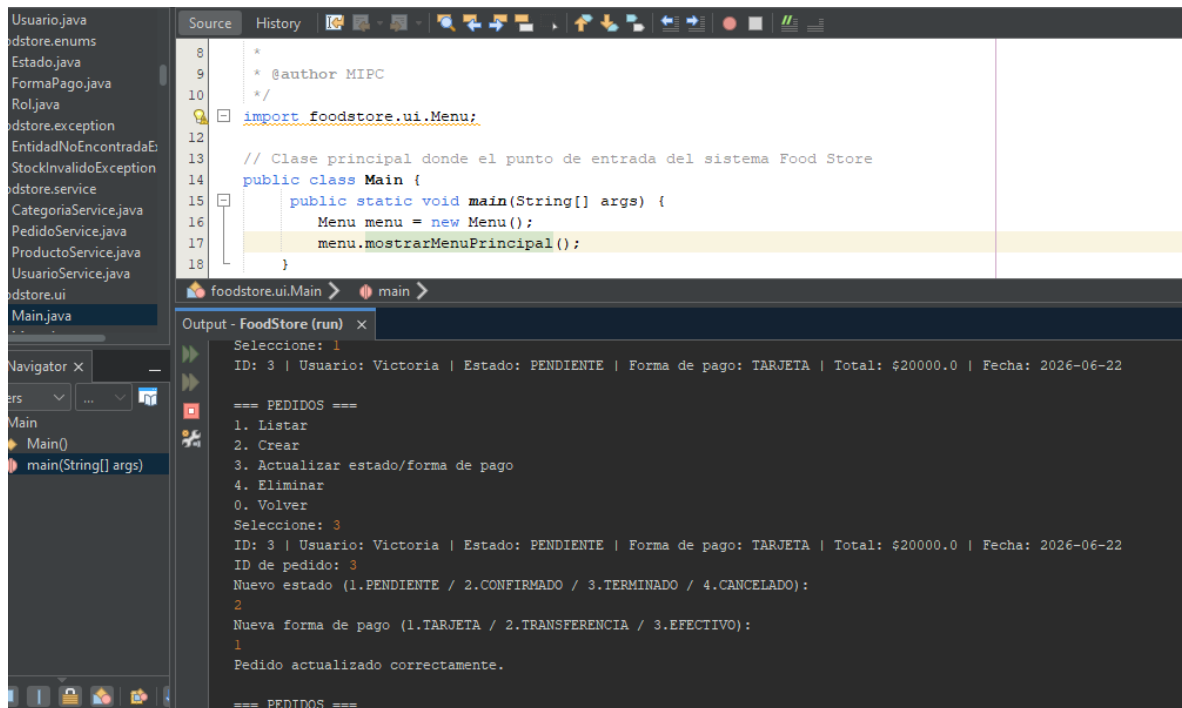
```
12 // Clase principal donde el punto de entrada del sistema Food Store
13
14 public class Main {
15     public static void main(String[] args) {
16         Menu menu = new Menu();
17         menu.mostrarMenuPrincipal();
18     }
19 }
20
```

Output - FoodStore (run) x

```
=== PEDIDOS ===
1. Listar
2. Crear
3. Actualizar estado/forma de pago
4. Eliminar
0. Volver
Seleccione: 1
ID: 3 | Usuario: Victoria | Estado: PENDIENTE | Forma de pago: TARJETA | Total: $20000.0 | Fecha: 2026-06-22

=== PEDIDOS ===
1. Listar
2. Crear
3. Actualizar estado/forma de pago
4. Eliminar
0. Volver
Seleccione:
```

Actualizar estado del pedido



The screenshot shows the same IDE with the `foodstore.ui.Main` class. The output window shows the menu options and the result of selecting option 3 (Actualizar estado/forma de pago). The output shows the current state of the order and the new state selected by the user.

```
8
9
10
11
12 // Clase principal donde el punto de entrada del sistema Food Store
13
14 public class Main {
15     public static void main(String[] args) {
16         Menu menu = new Menu();
17         menu.mostrarMenuPrincipal();
18     }
19 }
20
```

Output - FoodStore (run) x

```
Seleccione: 1
ID: 3 | Usuario: Victoria | Estado: PENDIENTE | Forma de pago: TARJETA | Total: $20000.0 | Fecha: 2026-06-22

=== PEDIDOS ===
1. Listar
2. Crear
3. Actualizar estado/forma de pago
4. Eliminar
0. Volver
Seleccione: 3
ID: 3 | Usuario: Victoria | Estado: PENDIENTE | Forma de pago: TARJETA | Total: $20000.0 | Fecha: 2026-06-22
ID de pedido: 3
Nuevo estado (1.PENDIENTE / 2.CONFIRMADO / 3.TERMINADO / 4.CANCELADO):
2
Nueva forma de pago (1.TARJETA / 2.TRANSFERENCIA / 3.EFECTIVO):
1
Pedido actualizado correctamente.

=== PEDIDOS ===
```

Eliminar pedido

```
8  *
9  * @author MIPC
10 * /
11
12 import foodstore.ui.Menu;
13
14 // Clase principal donde el punto de entrada del sistema Food Store
15 public class Main {
16     public static void main(String[] args) {
17         Menu menu = new Menu();
18         menu.mostrarMenuPrincipal();
19     }
20 }
```

Output - FoodStore (run) x

```
ID: 3 | Usuario: Victoria | Estado: PENDIENTE | Forma de pago: TARJETA | Total: $20000.0 | Fecha: 2026-06-22

=== PEDIDOS ===
1. Listar
2. Crear
3. Actualizar estado/forma de pago
4. Eliminar
0. Volver
Selecciones: 4
ID: 1 | Usuario: Victoria | Estado: PENDIENTE | Forma de pago: TARJETA | Total: $0.0 | Fecha: 2026-06-22
ID: 2 | Usuario: Victoria | Estado: PENDIENTE | Forma de pago: TARJETA | Total: $0.0 | Fecha: 2026-06-22
ID: 3 | Usuario: Victoria | Estado: PENDIENTE | Forma de pago: TARJETA | Total: $20000.0 | Fecha: 2026-06-22
ID a eliminar: 1
Confirma eliminacion? (S/N): S
Pedido eliminado correctamente.

=== PEDIDOS ===
1. Listar
2. Crear
```